

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT 24MEC51

EXERCISE NO: 11

GURUTHARSAN S	- 24MER011
HARISH KUMAR K K	- 24MER015
MADHANRAJ K	- 24MER028
SADAAGOPAAN T R	- 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 11

TITLE:

Design and develop a Wireless Environmental Parameter Monitoring System for Smart Industries. The system remotely monitors environmental conditions such as temperature, humidity, and air quality in industrial environments.

Done By:

GURTHARSAN S - 24MER011

HARISH KUMAR K K – 24MER015

MADHANRAJ K – 24MER028

SADAAGOPAAN T R – 24MER052

Project Overview:

The Wireless Environmental Parameter Monitoring System for Smart Industries is designed to continuously monitor important environmental conditions such as temperature, humidity, and air quality in industrial environments.

The system uses a DHT11 sensor to measure temperature and humidity and an MQ2 gas sensor to detect smoke and combustible gases. An ESP8266 NodeMCU acts as the main controller and provides wireless Wi-Fi connectivity.

The collected environmental data can be transmitted wirelessly to an IoT platform or monitoring dashboard. LEDs and a buzzer provide local warnings when the measured parameters exceed predefined prototype limits.

The system can be used for:

- Smart factories
- Manufacturing industries
- Workshops
- Warehouses
- Laboratories
- Industrial storage areas
- Industrial safety monitoring

Problem Statement

Industrial environments may experience changes in temperature, humidity, smoke, and harmful or combustible gases. If these conditions are not monitored continuously, they can affect worker safety, equipment performance, product quality, and overall industrial operation. Traditional monitoring systems may require manual inspection or separate instruments for each parameter.

The major problems are:

- Lack of continuous environmental monitoring.
- Difficulty in monitoring remote industrial areas.

- Delayed detection of abnormal conditions.
- Lack of wireless monitoring.
- Exposure of workers and equipment to unsafe environmental conditions.
- Difficulty in maintaining historical environmental data.

Therefore, a wireless environmental monitoring system is required to continuously measure important environmental parameters and provide real-time alerts.

Proposed Solution

The proposed system consists of:

- ESP8266 NodeMCU
- DHT11 temperature and humidity sensor
- MQ2 gas sensor
- Green, yellow, and red LEDs
- Buzzer
- Wi-Fi / IoT platform

The DHT11 measures temperature and humidity, while the MQ2 sensor provides an analog indication of smoke/combustible-gas presence. The ESP8266 processes the sensor values and sends the information wirelessly through Wi-Fi.

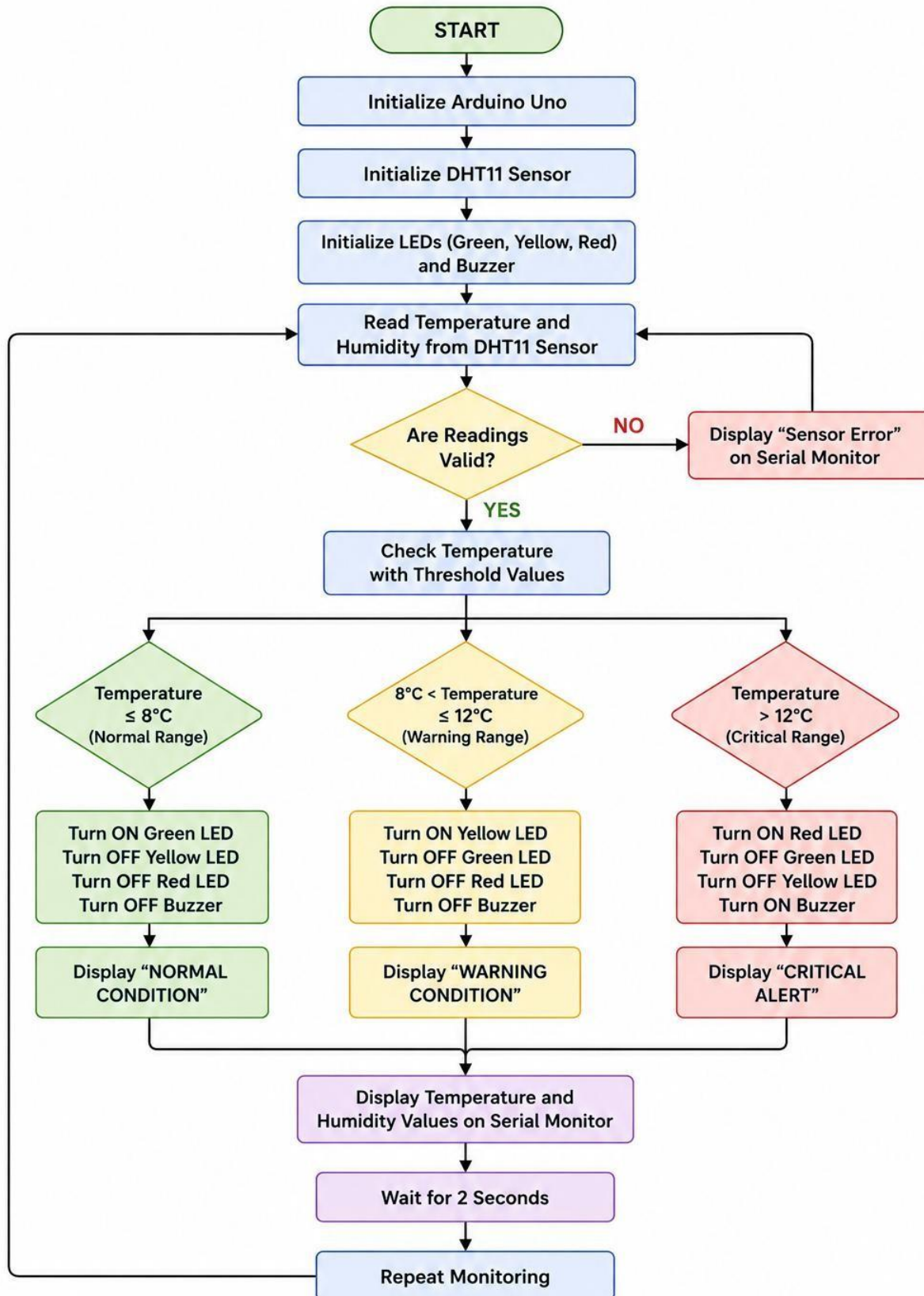
Prototype Environmental Conditions

Parameter	Normal	Warning	Critical
Temperature	$\leq 30^{\circ}\text{C}$	30–35°C	$> 35^{\circ}\text{C}$
Humidity	$\leq 60\%$	60–75%	$> 75\%$
MQ2 Sensor Value	≤ 300	301–600	> 600

These values are only prototype/simulation thresholds. MQ2 readings are ADC values, not calibrated ppm measurements. Real industrial safety systems require calibrated, gas-specific sensors and appropriate safety standards.

System Architecture

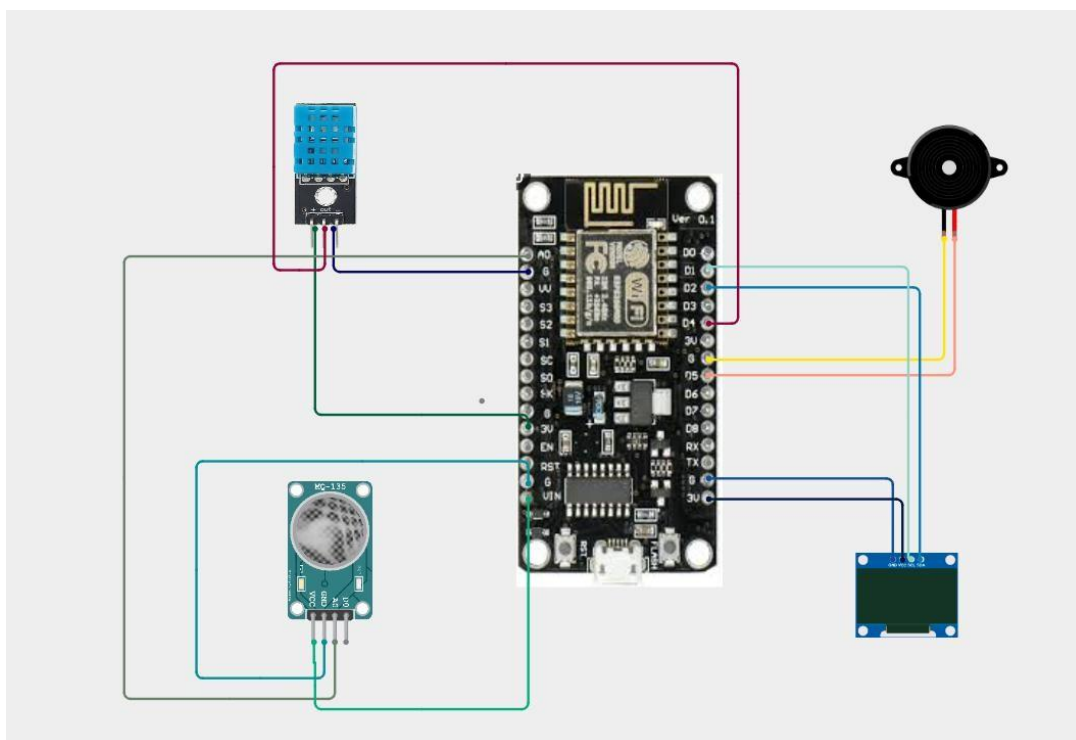
FLOWCHART SMART REFRIGERATOR MONITORING SYSTEM (Using DHT11 Sensor)



Circuit Table

S.No.	Component	Pin/Terminal	ESP8266 NodeMCU Connection
1	DHT11 Sensor	VCC	3.3V
2	DHT11 Sensor	GND	GND
3	DHT11 Sensor	DATA	D4 (GPIO2)
4	MQ2 Gas Sensor	VCC	5V/VU*
5	MQ2 Gas Sensor	GND	GND
6	MQ2 Gas Sensor	AO	A0**
7	Green LED	Anode	D1 (GPIO5) through 220 Ω
8	Green LED	Cathode	GND
9	Yellow LED	Anode	D2 (GPIO4) through 220 Ω
10	Yellow LED	Cathode	GND
11	Red LED	Anode	D5 (GPIO14) through 220 Ω
12	Red LED	Cathode	GND
13	Buzzer	Positive	D6 (GPIO12)
14	Buzzer	Negative	GND

Circuit Design



Algorithm

1. START
2. Initialize ESP8266 NodeMCU.
3. Initialize DHT11 Temperature and Humidity Sensor.
4. Initialize MQ2 Gas Sensor.
5. Initialize Green LED, Yellow LED, Red LED and Buzzer.
6. Connect ESP8266 to Wi-Fi.
7. Connect to the IoT monitoring platform.
8. Read temperature from the DHT11 sensor.
9. Read humidity from the DHT11 sensor.
10. Read analog gas value from the MQ2 sensor.
11. Check whether the sensor readings are valid.
12. Compare temperature, humidity and gas values with predefined thresholds.
13. If all parameters are within the normal range:
 - Turn ON Green LED.
 - Turn OFF Yellow LED.
 - Turn OFF Red LED.
 - Turn OFF Buzzer.
 - Display "NORMAL CONDITION".

14. Else if any parameter reaches the warning range:

Turn OFF Green LED.

Turn ON Yellow LED.

Turn OFF Red LED.

Turn OFF Buzzer.

Display "WARNING CONDITION".

15. Else if any parameter reaches the critical range:

Turn OFF Green LED.

Turn OFF Yellow LED.

Turn ON Red LED.

Turn ON Buzzer.

Display "CRITICAL CONDITION".

16. Send temperature, humidity and gas data
to the IoT dashboard through Wi-Fi.

17. Display the sensor values.

18. Wait for the next measurement interval.

19. Repeat the monitoring process continuously.

20. STOP

Coding

```
#include "AdafruitIO_WiFi.h"
```

```
#include <Wire.h>
```

```
#include <Adafruit_GFX.h>
```

```
#include <Adafruit_SSD1306.h>
```

```
#include <DHT.h>
```

```
//
```

```
=====
```

```
=
```

```
// WIFI DETAILS
```

```
//
```

```
=====
```

```
=
```

```
#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
```

```
#define WIFI_PASSWORD "qkts5737"
```

```
//
```

```
=====
```

```
=
```

```
// ADAFRUIT IO DETAILS
```

```
//
```

```
=====
```

```
=
```

```
#define IO_USERNAME  "Hair_13"
```

```
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
```

```
// Create Adafruit IO WiFi object
```

```
AdafruitIO_WiFi io(
```

```
IO_USERNAME,
```

```
IO_KEY,
```

```
WIFI_SSID,  
WIFI_PASSWORD  
);
```

```
//
```

```
=====
```

```
=
```

```
// FEEDS
```

```
//
```

```
=====
```

```
=
```

```
AdafruitIO_Feed *temperatureFeed =  
  io.feed("temperature");
```

```
AdafruitIO_Feed *humidityFeed =  
  io.feed("humidity");
```

```
AdafruitIO_Feed *mq135Feed =  
  io.feed("mq135");
```

```
AdafruitIO_Feed *airQualityAlertFeed =  
  io.feed("air-quality-alert");
```

```
//
```

```
=====
```

```
=
```

```
// DHT11
```

```
//
```

```
=====
```

```
=
```

```
#define DHT_PIN 4  
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
//
```

```
=====
```

```
=
```

```
// MQ135 AIR QUALITY SENSOR
```

```
//
```

```
=====
```

```
=
```

```
#define MQ135_PIN 34
```

```
//
```

```
=====
```

```
=
```

```
// BUZZER
```

```
//
```

```
=====
```

```
=
```

```
#define BUZZER_PIN 25
```

```
//
```

```
=====
```

```
=
```

```
// OLED
```

```
//
```

```
=====
```

```
=
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(  
  SCREEN_WIDTH,  
  SCREEN_HEIGHT,  
  &Wire,
```

```
);
```

```
//
```

```
=====
```

```
=
```

```
// VARIABLES
```

```
//
```

```
=====
```

```
=
```

```
float temperature = 0;
```

```
float humidity = 0;
```

```
int mql35Value = 0;
```

```
bool airQualityAlert = false;
```

```
//
```

```
=====
```

```
=
```

```
// TIMING
```

```
//
```

```
=====
```

```
=
```

```
unsigned long lastSend = 0;
```

```
const unsigned long sendInterval =  
    15000;
```

```
//
```

```
=====
```

```
=
```

```
// SETUP
```

```
//
```

```
=====
```

```
=
```

```
void setup()
{

    Serial.begin(9600);

    delay(1000);

    // -----
    // DHT
    // -----

    dht.begin();

    // -----
    // SENSOR PINS
    // -----

    pinMode(MQ135_PIN, INPUT);

    pinMode(BUZZER_PIN, OUTPUT);

    digitalWrite(
        BUZZER_PIN,
        LOW
    );

    // -----
    // OLED
    // -----

    Wire.begin(21, 22);

    if (!display.begin(
        SSD1306_SWITCHCAPVCC,
```

```
    0x3C))
{

    Serial.println(
        "OLED not detected!"
    );
}
else
{

    display.clearDisplay();

    display.setTextColor(
        SSD1306_WHITE
    );

    display.setTextSize(1);

    display.setCursor(10, 10);

    display.println(
        "SMART MONITOR"
    );

    display.setCursor(10, 30);

    display.println(
        "Starting..."
    );

    display.display();
}

// -----
// CONNECT TO ADAFRUIT IO
```

```
// -----  
  
Serial.println();  
Serial.println(  
    "Connecting to Adafruit IO..."  
);  
  
io.connect();  
  
// Wait for connection  
while (io.status() < AIO_CONNECTED)  
{  
  
    Serial.print(".");  
  
    delay(500);  
}  
  
Serial.println();  
  
Serial.println(  
    "Connected to Adafruit IO!"  
);  
  
display.clearDisplay();  
  
display.setTextSize(1);  
  
display.setCursor(0, 10);  
  
display.println(  
    "Adafruit IO"  
);  
  
display.setCursor(0, 30);
```

```
display.println(
  "Connected!"
);

display.display();

delay(2000);
}

//
=====

=
// LOOP
//
=====

=

void loop()
{

  // Keep Adafruit IO connection alive
  io.run();

  // -----
  // SEND SENSOR DATA
  // -----

  if (
    millis() - lastSend >=
    sendInterval
  )
  {

    lastSend = millis();
```



```
// =====  
// DHT11  
// =====  
  
temperature =  
    dht.readTemperature();  
  
humidity =  
    dht.readHumidity();  
  
if (isnan(temperature) ||  
    isnan(humidity))  
{  
  
    Serial.println(  
        "DHT11 reading failed!"  
    );  
  
}  
else  
{  
  
    Serial.print(  
        "Temperature: "  
    );  
  
    Serial.print(  
        temperature  
    );  
  
    Serial.println(" C");  
  
    Serial.print(  
        "Humidity: "
```

```
);

Serial.print(
  humidity
);

Serial.println(" %");
}

// =====
// MQ135
// =====

mq135Value =
  analogRead(MQ135_PIN);

Serial.print(
  "MQ135: "
);

Serial.println(
  mq135Value
);

// =====
// AIR QUALITY ALERT
// =====

// Adjust this threshold after testing

if (mq135Value > 2000)
{

  airQualityAlert = true;
```

```
digitalWrite(
  BUZZER_PIN,
  HIGH
);

}
else
{

  airQualityAlert = false;

  digitalWrite(
    BUZZER_PIN,
    LOW
  );
}

// =====
// SEND TO ADAFRUIT IO
// =====

Serial.println();

Serial.println(
  "Sending data to Adafruit IO..."
);

if (!isnan(temperature))
{

  temperatureFeed->save(
    temperature
  );
}
```

```
if (!isnan(humidity))
```

```
{
```

```
    humidityFeed->save(
```

```
        humidity
```

```
    );
```

```
}
```

```
mq135Feed->save(
```

```
    mq135Value
```

```
);
```

```
airQualityAlertFeed->save(
```

```
    airQualityAlert ? 1 : 0
```

```
);
```

```
Serial.println(
```

```
    "Data sent successfully!"
```

```
);
```

```
// =====
```

```
// OLED
```

```
// =====
```

```
display.clearDisplay();
```

```
display.setTextColor(
```

```
    SSD1306_WHITE
```

```
);
```

```
display.setTextSize(1);
```

```
display.setCursor(0, 0);
```

```
display.print("T:");
```

```
display.print(
  temperature,
  1
);

display.print("C ");

display.print("H:");

display.print(
  humidity,
  0
);

display.println("%");

display.setCursor(0, 15);

display.print("MQ135: ");

display.println(
  mq135Value
);

display.setCursor(0, 30);

if (airQualityAlert)
{

  display.println(
    "AIR QUALITY ALERT!"
  );

}
```

```
else
{

    display.println(
        "Air Quality Normal"
    );
}

display.setCursor(0, 45);

display.print("WiFi: ");

display.println(
    "Connected"
);

display.display();

Serial.println(
    "=====")
);
}
}
#include "AdafruitIO_WiFi.h"

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include <DHT.h>

//
=====
=
// WIFI DETAILS
```

```
//
```

```
=====
```

```
=
```

```
#define WIFI_SSID    "Harish Kumar K K"
```

```
#define WIFI_PASSWORD "Mavboiii@10"
```

```
//
```

```
=====
```

```
=
```

```
// ADAFRUIT IO DETAILS
```

```
//
```

```
=====
```

```
=
```

```
#define IO_USERNAME  "Hair_13"
```

```
#define IO_KEY       "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"
```

```
// Create Adafruit IO WiFi object
```

```
AdafruitIO_WiFi io(
```

```
IO_USERNAME,
```

```
IO_KEY,
```

```
WIFI_SSID,
```

```
WIFI_PASSWORD
```

```
);
```

```
//
```

```
=====
```

```
=
```

```
// FEEDS
```

```
//
```

```
=====
```

```
=
```

```
AdafruitIO_Feed *temperatureFeed =
```

```
io.feed("temperature");
```

```
AdafruitIO_Feed *humidityFeed =  
  io.feed("humidity");
```

```
AdafruitIO_Feed *mq135Feed =  
  io.feed("mq135");
```

```
AdafruitIO_Feed *airQualityAlertFeed =  
  io.feed("air-quality-alert");
```

```
//
```

```
=====
```

```
=
```

```
// DHT11
```

```
//
```

```
=====
```

```
=
```

```
#define DHT_PIN 4
```

```
#define DHT_TYPE DHT11
```

```
DHT dht(DHT_PIN, DHT_TYPE);
```

```
//
```

```
=====
```

```
=
```

```
// MQ135 AIR QUALITY SENSOR
```

```
//
```

```
=====
```

```
=
```

```
#define MQ135_PIN 34
```

```
//
```

```
=====
```

```
=
```



```
// BUZZER
```

```
//
```

```
=====
```

```
=
```

```
#define BUZZER_PIN 25
```

```
//
```

```
=====
```

```
=
```

```
// OLED
```

```
//
```

```
=====
```

```
=
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(
```

```
  SCREEN_WIDTH,
```

```
  SCREEN_HEIGHT,
```

```
  &Wire,
```

```
  -1
```

```
);
```

```
//
```

```
=====
```

```
=
```

```
// VARIABLES
```

```
//
```

```
=====
```

```
=
```

```
float temperature = 0;
```

```
float humidity = 0;
```

```
int mq135Value = 0;
```

```
bool airQualityAlert = false;
```

```
//
```

```
=====
```

```
=
```

```
// TIMING
```

```
//
```

```
=====
```

```
=
```

```
unsigned long lastSend = 0;
```

```
const unsigned long sendInterval =  
    15000;
```

```
//
```

```
=====
```

```
=
```

```
// SETUP
```

```
//
```

```
=====
```

```
=
```

```
void setup()
```

```
{
```

```
    Serial.begin(9600);
```

```
    delay(1000);
```

```
    // -----
```

```
    // DHT
```

```
    // -----
```

```
dht.begin();

// -----
// SENSOR PINS
// -----

pinMode(MQ135_PIN, INPUT);

pinMode(BUZZER_PIN, OUTPUT);

digitalWrite(
  BUZZER_PIN,
  LOW
);

// -----
// OLED
// -----

Wire.begin(21, 22);

if (!display.begin(
  SSD1306_SWITCHCAPVCC,
  0x3C))
{

  Serial.println(
    "OLED not detected!"
  );
}
else
{

  display.clearDisplay();
```

```
display.setTextColor(
  SSD1306_WHITE
);

display.setTextSize(1);

display.setCursor(10, 10);

display.println(
  "SMART MONITOR"
);

display.setCursor(10, 30);

display.println(
  "Starting..."
);

display.display();
}

// -----
// CONNECT TO ADAFRUIT IO
// -----

Serial.println();
Serial.println(
  "Connecting to Adafruit IO..."
);

io.connect();

// Wait for connection
while (io.status() < AIO_CONNECTED)
{
```

```
Serial.print(".");

delay(500);
}

Serial.println();

Serial.println(
  "Connected to Adafruit IO!"
);

display.clearDisplay();

display.setTextSize(1);

display.setCursor(0, 10);

display.println(
  "Adafruit IO"
);

display.setCursor(0, 30);

display.println(
  "Connected!"
);

display.display();

delay(2000);
}

//
```

```

=
// LOOP
//
=====

=

void loop()
{

    // Keep Adafruit IO connection alive
    io.run();

    // -----
    // SEND SENSOR DATA
    // -----

    if (
        millis() - lastSend >=
        sendInterval
    )
    {

        lastSend = millis();

        // =====
        // DHT11
        // =====

        temperature =
            dht.readTemperature();

        humidity =
            dht.readHumidity();

        if (isnan(temperature) ||

```

```
    isnan(humidity))
{

    Serial.println(
        "DHT11 reading failed!"
    );

}
else
{

    Serial.print(
        "Temperature: "
    );

    Serial.print(
        temperature
    );

    Serial.println(" C");

    Serial.print(
        "Humidity: "
    );

    Serial.print(
        humidity
    );

    Serial.println(" %");
}
```

```
// =====
// MQ135
// =====
```

```
mq135Value =  
  analogRead(MQ135_PIN);
```

```
Serial.print(  
  "MQ135: "  
);
```

```
Serial.println(  
  mq135Value  
);
```

```
// =====  
// AIR QUALITY ALERT  
// =====
```

```
// Adjust this threshold after testing
```

```
if (mq135Value > 2000)  
{
```

```
  airQualityAlert = true;
```

```
  digitalWrite(  
    BUZZER_PIN,  
    HIGH  
  );
```

```
}
```

```
else
```

```
{
```

```
  airQualityAlert = false;
```

```
  digitalWrite(  
    BUZZER_PIN,  
    LOW  
  );
```



```
        BUZZER_PIN,
        LOW
    );
}

// =====
// SEND TO ADAFRUIT IO
// =====

Serial.println();

Serial.println(
    "Sending data to Adafruit IO..."
);

if (!isnan(temperature))
{

    temperatureFeed->save(
        temperature
    );
}

if (!isnan(humidity))
{

    humidityFeed->save(
        humidity
    );
}

mq135Feed->save(
    mq135Value
);
```

```
airQualityAlertFeed->save(  
    airQualityAlert ? 1 : 0  
);
```

```
Serial.println(  
    "Data sent successfully!"  
);
```

```
// =====  
// OLED  
// =====
```

```
display.clearDisplay();
```

```
display.setTextColor(  
    SSD1306_WHITE  
);
```

```
display.setTextSize(1);
```

```
display.setCursor(0, 0);
```

```
display.print("T:");
```

```
display.print(  
    temperature,  
    1  
);
```

```
display.print("C ");
```

```
display.print("H:");
```

```
display.print(  
    humidity,
```

```
    0
);

display.println("%");

display.setCursor(0, 15);

display.print("MQ135: ");

display.println(
    mq135Value
);

display.setCursor(0, 30);

if (airQualityAlert)
{

    display.println(
        "AIR QUALITY ALERT!"
    );

}
else
{

    display.println(
        "Air Quality Normal"
    );
}

display.setCursor(0, 45);

display.print("WiFi: ");
```

```
display.println(  
  "Connected"  
);  
  
display.display();  
  
Serial.println(  
  "===== "  
);  
}  
}
```

System Working

The system is installed in the industrial environment where environmental parameters need to be monitored.

The DHT11 sensor continuously measures temperature and humidity. At the same time, the MQ2 sensor monitors smoke and combustible-gas presence.

The ESP8266 receives these readings and compares them with the predefined threshold values.

Normal Condition

When temperature, humidity, and gas readings are within the normal prototype range:

Green LED → ON

Buzzer → OFF

The system reports a normal environmental condition.

Warning Condition

When one or more parameters begin to exceed the normal range:

Yellow LED → ON

The system indicates that attention may be required.

Critical Condition

When a parameter exceeds the critical prototype threshold:

Red LED → ON

Buzzer → ON

The system generates an immediate local warning.

The ESP8266 can simultaneously transmit the environmental data wirelessly to an IoT dashboard, allowing authorized personnel to monitor the industrial environment remotely.

Bill of Materials

S.No.	Component	Specification	Quantity	Purpose
1	ESP8266 NodeMCU	Wi-Fi Microcontroller	1	Main Controller & Wireless Communication
2	DHT11 Sensor	Temperature & Humidity	1	Environmental Monitoring
3	MQ2 Gas Sensor	Smoke/Combustible Gas	1	Air Quality/Gas Monitoring
4	Green LED	5 mm	1	Normal Indication
5	Yellow LED	5 mm	1	Warning Indication
6	Red LED	5 mm	1	Critical Indication
7	Resistor	220 Ω	3	LED Current Limiting
8	Buzzer	3.3/5 V	1	Audible Alert
9	Breadboard	Standard	1	Circuit Assembly
10	Jumper Wires	Male-Male	1 Set	Circuit Connections
11	USB Cable	Micro USB	1	Power & Programming

Prototype Implementation

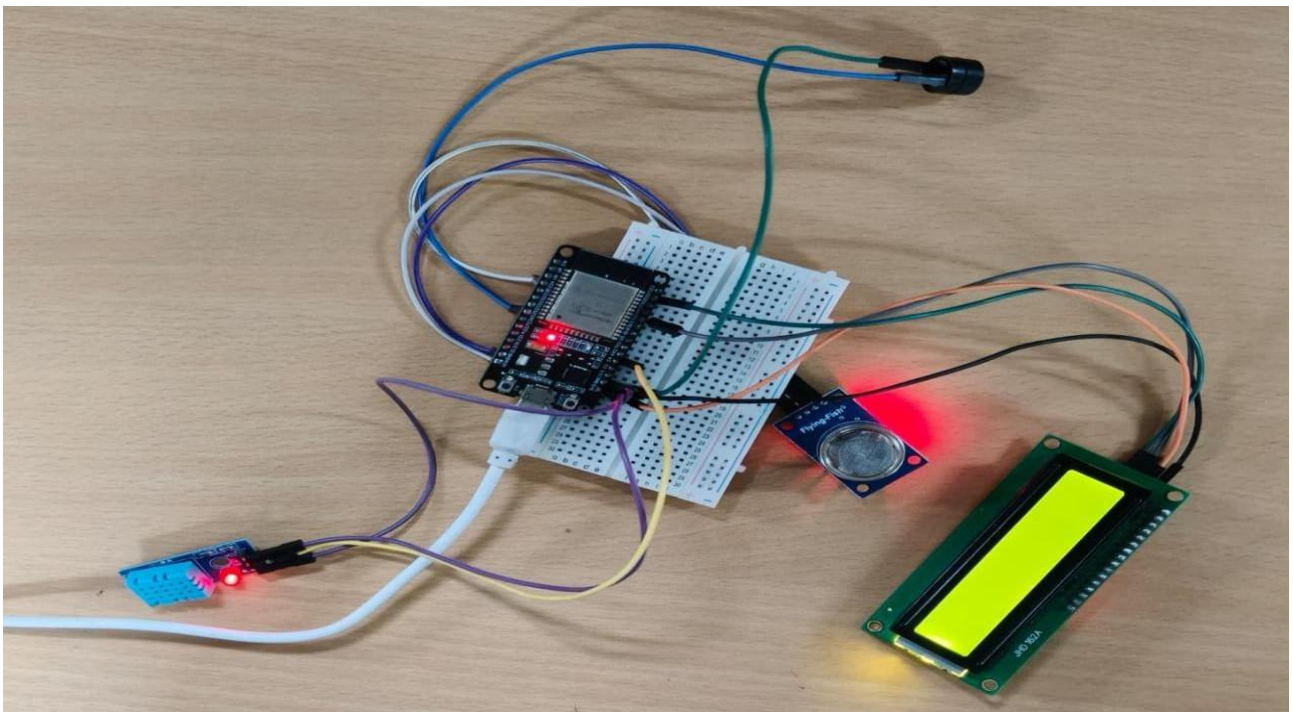
The prototype can be assembled on a breadboard using the ESP8266 NodeMCU, DHT11, MQ2, LEDs, buzzer, and jumper wires.

The DHT11 sensor is positioned in the area where temperature and humidity need to be monitored. The MQ2 sensor is positioned so that it can sample the surrounding air.

The ESP8266 is powered through USB and programmed using the Arduino IDE.

After startup, the ESP8266 reads the sensors at regular intervals. The environmental readings are displayed on the Serial Monitor and can be transmitted wirelessly to an IoT dashboard.

The LEDs provide local status indication, while the buzzer provides an audible critical alert.



Results & Performance Analysis

The proposed system successfully demonstrates simultaneous monitoring of multiple environmental parameters.

Parameter	Result
Controller	ESP8266 NodeMCU
Temperature Monitoring	Yes
Humidity Monitoring	Yes
Gas/Smoke Monitoring	Yes
Wireless Communication	Wi-Fi
Remote Monitoring	Supported
Normal Alert	Green LED
Warning Alert	Yellow LED
Critical Alert	Red LED + Buzzer
Continuous Monitoring	Yes

Performance Analysis

The system provides a low-cost method of monitoring multiple environmental parameters from a single wireless node.

The ESP8266 allows environmental readings to be transmitted remotely without requiring a separate Wi-Fi module.

The multi-level alert system makes it easy for operators to identify environmental conditions quickly.

However, the MQ2 is a broad-response gas sensor and should not be treated as a calibrated industrial air-quality instrument. For real industrial deployment, certified and gas-specific sensors should be selected according to the hazard being monitored.

